

Controle de Hardware com pyFirmata2

Integração de Arduino com Python e VS Code

Karina Casola

Unisul

Projetos Básicos

Projeto 1: Pisca LED

Projeto 2: Alarme com LED e Buzzer

Projeto 3: Controle Interativo por Teclado

Projeto 4: Lógicas de Entrada com Botões

Projeto 5: Simulação de Semáforo Completo

Projeto 6: Termômetro com Alerta Sonoro

Projeto 07: Sensor de Luminosidade (LDR)

Metas e Objetivos da oficina

A Meta Principal

Capacitar o aluno a integrar o ecossistema de software **Python** com o hardware de microcontroladores de forma rápida, interativa e eficiente.

Ao final desta oficina, você será capaz de:

- **Compreender** a arquitetura de comunicação entre o computador (Lógica) e a placa (Aquisição de Dados).
- **Preparar** o microcontrolador carregando o firmware correto (*StandardFirmata*).
- **Configurar** um ambiente de desenvolvimento profissional integrado ao VS Code.
- **Desenvolver e executar** scripts em Python para controlar portas digitais.

Projetos Básicos

Projeto 1: Pisca LED

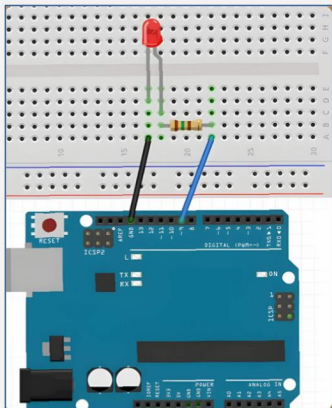
Projeto 1: Pisca LED

Objetivo Central: Montar um circuito básico na protoboard com o objetivo de fazer um LED piscar.

Sobre a Atividade

Trata-se de uma **atividade introdutória**, o pontapé inicial ideal para que você possa começar a compreender:

- Os fundamentos da **programação no Arduino** (e como aplicaremos isso no Python).
- A montagem e a conexão correta dos **componentes eletrônicos** na protoboard.



	01	Placa Uno com cabo usb
	01	Protoboard
	01	Led
	01	Resistor 150R (marrom, verde, marrom, dourado)
	02	Jumper macho x macho

Projeto 1: Pisca LED (De C++ para Python)

Este é o seu projeto inicial! Vamos acionar um componente físico usando Python.

Entendendo a Tradução do Código

A lógica de programação é a mesma, mas os comandos mudam:

- **A função `setup()`** vira a inicialização de variáveis no topo do nosso script em Python.
- **A função `loop()`** é substituída por um laço infinito usando `while True`:
- `digitalWrite(led, HIGH)` se transforma em `placa.digital[led].write(1)`.
- `delay(1000)` é gerenciado pela biblioteca de tempo do Python como `time.sleep(1)` (em segundos, não milissegundos).

Projeto 1: Código Fonte

Objetivo: Fazer um LED piscar no Pino 9 da placa.

```
from pyfirmata2 import Arduino
import time

PORTA = 'COM3' # Altere para a sua porta COM
placa = Arduino(PORTA)
led = 9 # Variavel para o pino 9

print("Iniciando Pisca LED... Pressione Ctrl+C para parar.")

# try/except garante fechamento seguro
try:
    while True: # Loop infinito
        placa.digital[led].write(1) # HIGH (Liga)
        time.sleep(1) # Pausa de 1 seg

        placa.digital[led].write(0) # LOW (Desliga)
        time.sleep(1) # Pausa de 1 seg

except KeyboardInterrupt:
    placa.exit() # Encerra a conexao
```

Projeto 2: Alarme com LED e Buzzer

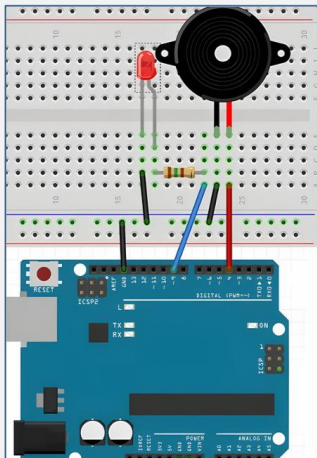
Projeto 2: Pisca LED + Buzzer

Objetivo Central: Montar um circuito simples na protoboard integrando um LED e um buzzer para criar um alarme básico.

Sobre a Atividade

Neste exercício prático, daremos um passo além e expandiremos nossos conhecimentos:

- **Funcionamento:** Quando ativado, o circuito fará com que o LED acenda e o buzzer emita som **simultaneamente**.
- **Aprendizado:** É a prática ideal para aprender a ligar **múltiplos componentes** eletrônicos e programar o acionamento de várias saídas da placa ao mesmo tempo.



	01	Placa Uno com cabo usb
	01	Protoboard
	01	Led
	01	Resistor 150R (marrom, verde, marrom, dourado)
	01	Buzzer
	05	Jumper macho x macho

Objetivos do Projeto 2

Neste projeto, evoluímos para o controle de múltiplos atuadores simultâneos.

A Meta

Criar um sistema de alarme visual e sonoro onde um LED e um Buzzer operam em total sincronia utilizando o protocolo Firmata.

Conceitos trabalhados:

- **Paralelismo Lógico:** Acionar múltiplas portas antes de executar uma pausa (*sleep*).
- **Gestão de Corrente:** Compreender a conexão de dois componentes na protoboard dividindo o barramento de energia.
- **Mapeamento de Pinos:** Uso dos pinos digitais 9 (LED) e 4 (Buzzer).

Projeto 2: Código Fonte (pyfirmata2)

Objetivo: Sincronizar LED (Pino 9) e Buzzer (Pino 4).

```
from pyfirmata2 import Arduino
import time

PORTA = 'COM3' # Verifique sua porta no Gerenciador de Dispositivos
placa = Arduino(PORTA)

# Definicao dos pinos
led = 9
buzzer = 4

print("Alarme Ativo! Pressione Ctrl+C para encerrar.")

try:
    while True:
        # LIGA os dois componentes simultaneamente
        placa.digital[led].write(1)
        placa.digital[buzzer].write(1)
        time.sleep(1) # Pausa com ambos ligados

        # DESLIGA os dois componentes simultaneamente
        placa.digital[led].write(0)
        placa.digital[buzzer].write(0)
        time.sleep(1) # Pausa com ambos desligados

except KeyboardInterrupt:
    placa.exit()
    print("Sistema desativado.")
```

Diferenças Técnicas Importantes

- **No Arduino (C++):** O comando `delay(1000)` interrompe todo o processamento do microcontrolador por 1 segundo.
- **No Python:** O `time.sleep(1)` interrompe a execução do seu script no computador, mas a placa continua mantendo o estado (HIGH ou LOW) enviado anteriormente.
- **Sincronia:** Como a comunicação serial é muito rápida, enviar o comando para o pino 9 e depois para o pino 4 parece instantâneo para o ouvido e olho humano, criando o efeito de alarme.

Projeto 3: Controle Interativo por Teclado

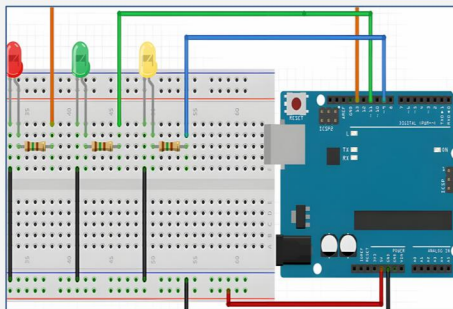
Projeto 3: Controle Interativo

Objetivo Central: Controlar LEDs (ou outros atuadores) através de comandos enviados pelo teclado do computador.

Sobre a Atividade

Neste projeto, exploramos a capacidade da placa de receber informações externas e processá-las em tempo real:

- **Interatividade:** O sistema reage de forma imediata aos comandos digitados pelo usuário no terminal.
- **Versatilidade:** Embora utilizemos LEDs, a lógica de programação é a mesma para acionar **lâmpadas, relés ou motores**.
- **Fluxo de Dados:** Entender como o Python lê o *input* do teclado e o traduz em um sinal elétrico na placa.



	01	Placa Uno com cabo usb
	01	Protoboard
	03	Led (vermelho, amarelo, verde)
	03	Resistor 150R (marrom, verde, marrom, dourado)
	08	Jumper macho x macho

Projeto 3: Código Fonte (Parte 1)

Objetivo: Controlar 3 LEDs (Amarelo, Verde, Vermelho) via teclado.

```
from pyfirmata2 import Arduino
import time

placa = Arduino('COM3')

# Mapeamento dos Pinos
led_amarelo = 9
led_verde = 11
led_vermelho = 13

print("Comandos: [y] Amarelo | [g] Verde | [r] Vermelho | [s] Sair")

try:
    while True:
        comando = input("Digite um comando: ").lower()

        if comando == 'y':
            placa.digital[led_amarelo].write(1)
            print("LED AMARELO LIGADO. Desligando em 3s...")
            for i in range(3, 0, -1):
                print(f"... {i}")
                time.sleep(1)
            placa.digital[led_amarelo].write(0)
            print("LED AMARELO DESLIGADO")

        elif comando == 'g':
            placa.digital[led_verde].write(1)
            print("LED VERDE LIGADO")
            time.sleep(2)
            placa.digital[led_verde].write(0)
            print("LED VERDE DESLIGADO")
```

Projeto 3: Código Fonte (Parte 2)

```
elif comando == 'r':
    placa.digital[led_vermelho].write(1)
    print("LED VERMELHO LIGADO")
    time.sleep(2)
    placa.digital[led_vermelho].write(0)
    print("LED VERMELHO DESLIGADO")

elif comando == 's':
    print("Saindo do programa...")
    break

else:
    print("Comando invalido!")

except KeyboardInterrupt:
    pass

placa.exit()
```

Destaque Tecnico

Observe que no Python usamos o `input()` para pausar o código e aguardar o usuário, eliminando a necessidade de verificar se a Serial está disponível (*Serial.available*).

Comparativo de Fluxo: C++ vs Python

- **No Arduino (C++):** O programa roda o `loop()` freneticamente e você precisa checar se algo "chegou" na Serial.
- **No Python:** O programa "estaciona" na linha do `input()` e só continua após você digitar e apertar Enter.
- **Estrutura:** O `switch/case` do C++ foi substituído por uma estrutura `if/elif/else`, que é mais comum e legível em Python.
- **Temporizador:** No exemplo do LED Amarelo, usamos um laço `for` para criar a contagem regressiva de forma elegante.

Projeto 4: Lógicas de Entrada com Botões

Projeto 4: Push Buttons + LEDs

Objetivo Central: Controlar LEDs através de botões pulsadores, explorando diferentes lógicas de programação e tipos de acionamento.

Dinâmicas de Interação

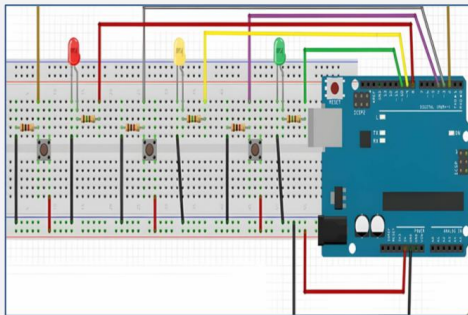
Neste projeto, veremos como o mesmo componente (botão) pode executar funções distintas:

Exemplo 1 (Vermelho): Temporizador – Acende ao apertar e desliga sozinho após um tempo.

Exemplo 2 (Amarelo): Evento – Inicia uma sequência de pisca-pisca após um único pulso.

Exemplo 3 (Verde): Estado Real – Permanece aceso apenas enquanto o botão estiver fisicamente pressionado.

- **Foco do Aprendizado:** Leitura de portas digitais de entrada (*Digital Input*) e diferenciação entre pulso e retenção.



	01	Placa Uno com cabo usb
	01	Protoboard
	03	Led
	03	Resistor 150R (marrom, verde, marrom, dourado)
	03	Push button
	03	Resistor 1K (marrom, preto, vermelho, dourado)
	17	Jumper macho x macho

Projeto 4: Configuração de Entradas

Para ler botões no **pyfirmata2**, precisamos ativar o sistema de amostragem da placa.

```
from pyfirmata2 import Arduino
import time

placa = Arduino('COM3')

# Definicao dos Pinos (Botoes como INPUT e LEDs como OUTPUT)
btn_vermelho = placa.get_pin('d:2:i') # d=digital, 2=pino, i=input
btn_amarelo = placa.get_pin('d:3:i')
btn_verde = placa.get_pin('d:4:i')

led_vermelho = placa.get_pin('d:8:o') # o=output
led_amarelo = placa.get_pin('d:9:o')
led_verde = placa.get_pin('d:10:o')

# Habilita a leitura dos pinos de entrada
placa.samplingOn()
time.sleep(0.1) # Pequena pausa para estabilizar a leitura

print("Sistema pronto. Pressione os botoes!")
```

Projeto 4: Loop de Execução (Parte 1)

```
try:
    while True:
        # EXEMPLO 1: LED VERMELHO (Temporizador)
        if btn_vermelho.read() is True:
            print("Botao Vermelho: LED ligado por 5s")
            led_vermelho.write(1)
            time.sleep(5)
            led_vermelho.write(0)

        # EXEMPLO 2: LED AMARELO (Evento de Pisca)
        if btn_amarelo.read() is True:
            print("Botao Amarelo: Iniciando sequencia de pisca")
            for i in range(5):
                led_amarelo.write(1)
                time.sleep(0.5)
                led_amarelo.write(0)
                time.sleep(0.5)
```

Projeto 4: Loop de Execução (Parte 2)

```
# EXEMPLO 3: LED VERDE (Estado Real / Acende enquanto pressiona)
if btn_verde.read() is True:
    led_verde.write(1)
else:
    led_verde.write(0)

time.sleep(0.01) # Pausa minima para nao sobrecarregar o processador

except KeyboardInterrupt:
    placa.exit()
    print("Programa finalizado.")
```

Nota sobre Entradas Digitais

No Python, usamos `btn.read() is True`. Note que o `pyfirmata2` retorna `True/False` ou `None` (se a leitura ainda não foi estabilizada), por isso a pausa de 0.1s no início é fundamental.

Principais Diferenças na Leitura de Botões

- **Sintaxe de Pino:** Usamos `get_pin('d:2:i')` que já define o pino 2 como Digital e Input em uma única linha.
- **Sampling:** O comando `placa.samplingOn()` é obrigatório. Sem ele, o método `.read()` sempre retornará `None`.
- **Estabilidade:** Adicionamos um `time.sleep(0.01)` no final do loop. No Arduino C++, o `delay` é opcional, mas no Python ele evita que o seu computador use 100% da CPU tentando ler o botão bilhões de vezes por segundo.
- **Lógica de Bloqueio:** No Exemplo 1 e 2, o programa "trava" nos *sleeps* de espera. Enquanto o LED vermelho espera os 5 segundos, o botão verde não funcionará. (Para resolver isso, seriam necessários conceitos avançados de *Threading*).

Projeto 5: Simulação de Semáforo Completo

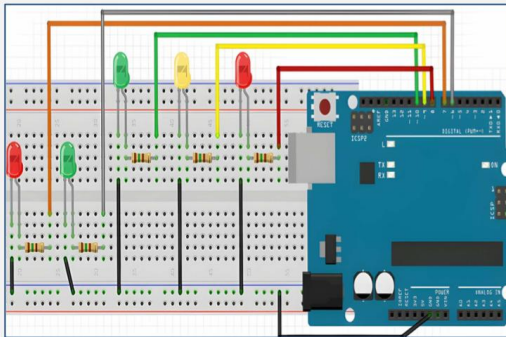
Projeto 5: Semáforo com Sinal de Pedestres

Objetivo Central: Construir um sistema de sinalização viária completo, simulando o funcionamento real de um cruzamento com foco em pedestres.

Sobre a Atividade

Neste projeto, aplicaremos a lógica de controle temporal para gerenciar um conjunto maior de atuadores:

- **Simulação Realista:** Controle preciso do tempo de acendimento dos LEDs (Verde, Amarelo e Vermelho para carros; Verde e Vermelho para pedestres).
- **Lógica de Sequenciamento:** Programação de estados lógicos dependentes, onde um sinal só abre quando o outro obrigatoriamente fecha.
- **Complexidade Gradual:** Exercício fundamental para dominar o controle de múltiplos LEDs e variáveis de tempo.



	01	Placa Uno com cabo usb
	01	Protoboard
	05	Led (02x vermelho, 01x amarelo, 02x verde)
	05	Resistor 150R (marrom, verde, marrom, dourado)
	11	Jumper macho x macho

Projeto 5: Configuração do Hardware

Objetivo: Gerenciar 5 saídas digitais em sincronia.

```
from pyfirmata2 import Arduino
import time

placa = Arduino('COM3')

# Veiculos
led_vermelho = placa.get_pin('d:8:o')
led_amarelo = placa.get_pin('d:9:o')
led_verde = placa.get_pin('d:10:o')

# Pedestres
ped_verde = placa.get_pin('d:6:o')
ped_vermelho = placa.get_pin('d:7:o')

print("Semáforo em operação... Pressione Ctrl+C para encerrar.")
```

Projeto 5: Ciclo do Semáforo

```
try:
    while True:
        # FASE 1: Verde Veiculos / Vermelho Pedestres
        led_verde.write(1)
        ped_vermelho.write(1)
        time.sleep(5)

        # FASE 2: Amarelo Veiculos
        led_verde.write(0)
        time.sleep(1)
        ped_vermelho.write(0)
        time.sleep(0.2)
        led_amarelo.write(1)
        time.sleep(1)

        # FASE 3: Vermelho Veiculos / Verde Pedestres
        led_amarelo.write(0)
        time.sleep(0.5)
        led_vermelho.write(1)
        ped_verde.write(1)
        time.sleep(5)

        # Reset para Reinicio
        led_vermelho.write(0)
        time.sleep(1)
        ped_verde.write(0)

except KeyboardInterrupt:
    placa.exit()
```

- **Segurança:** Note as pequenas pausas (*sleeps*) entre as trocas de sinal. Elas simulam o tempo de segurança real para evitar colisões.
- **Manutenibilidade:** No Python, poderíamos criar uma **função** para cada estado do semáforo (ex: `fase_pedestres()`), tornando o código ainda mais organizado que no C++.
- **Estado Padrão:** Diferente do Arduino que reinicia o loop imediatamente, no final do nosso `while`, garantimos que os LEDs apaguem antes da próxima iteração para evitar sobreposição de cores.

Projeto 6: Termômetro com Alerta Sonoro

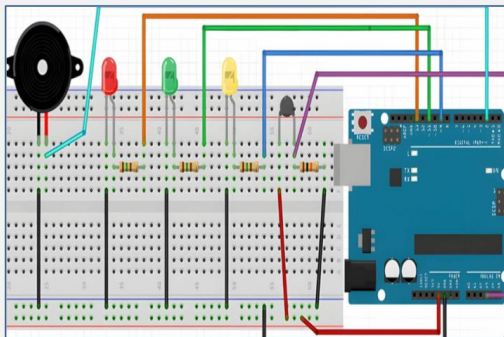
Projeto 6: Medidor de Temperatura

Objetivo Central: Criar um sistema de monitoramento térmico usando um termistor NTC 10K, com alertas visuais e sonoros baseados em faixas de temperatura.

Lógica de Alerta (Feedback)

O sistema deve reagir conforme a temperatura lida no ambiente:

- **Frio ($\leq 20^{\circ}\text{C}$):** Apenas o LED Verde acende.
- **Confortável ($\leq 30^{\circ}\text{C}$):** O LED Amarelo acende.
- **Quente ($\geq 31^{\circ}\text{C}$):** O LED Vermelho acende e o Buzzer é ativado.
- **Monitoramento:** Utilizaremos o terminal do Python para exibir as leituras de temperatura em tempo real, permitindo a calibração do sistema.



	01	Placa Uno com cabo usb
	01	Protoboard
	03	Led (vermelho, amarelo, verde)
	03x - Resistor 150R (marrom, verde, marrom, dourado) 01x - Resistor 10K (marrom, preto, laranja, dourado)	
	01	Termistor NTC 10k (sensor de temperatura)
	11	Jumper macho x macho
	1	Buzzer

Projeto 6: Lógica e Conversão Analógica

Diferente do Arduino C++, no Python convertemos o sinal analógico (0.0 a 1.0) manualmente para graus Celsius.

```
from pyfirmata2 import Arduino
import time
import math

placa = Arduino('COM3')
sensor = placa.get_pin('a:3:i') # Pino Analogico A3
placa.samplingOn()

led_v = placa.get_pin('d:13:o') # Vermelho
led_a = placa.get_pin('d:9:o')  # Amarelo
led_g = placa.get_pin('d:11:o') # Verde
buzzer = placa.get_pin('d:2:o')

def get_celsius(valor_bruto):
    if valor_bruto is None or valor_bruto == 0: return 0
    # Conversao simplificada para NTC 10k
    resistencia = (1 / valor_bruto) - 1
    temp = 1 / (math.log(resistencia) / 3950 + 1 / 298.15) - 273.15
    return round(temp, 1)
```

Projeto 6: Loop de Monitoramento

```
try:
    while True:
        t = get_celsius(sensor.read())
        print(f"Temperatura Atual: {t}*C")

        if t <= 20: # FRIJO
            led_g.write(1); led_a.write(0); led_v.write(0); buzzer.write(0)

        elif t <= 30: # CONFORTO
            led_a.write(1); led_g.write(0); led_v.write(0); buzzer.write(0)

        elif t >= 31: # QUENTE
            led_v.write(1); buzzer.write(1); led_g.write(0); led_a.write(0)

        time.sleep(1) # Atualiza a cada 1 segundo

except KeyboardInterrupt:
    placa.exit()
```

Equivalência Tecnológica

A biblioteca **Thermistor.h** faz o cálculo logarítmico "escondido". No Python, usamos a biblioteca **math** para ter controle total sobre a precisão da leitura.

Projeto 07: Sensor de Luminosidade (LDR)

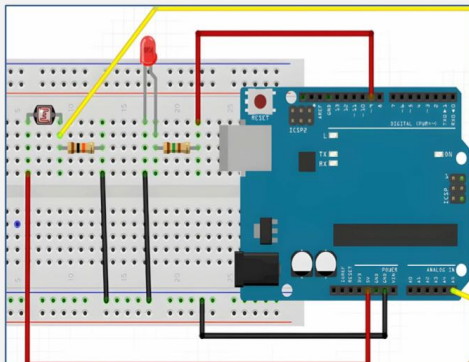
Projeto 07: Sensor de Luminosidade (LDR)

Objetivo Central: Criar um sistema de iluminação automática que reage à claridade do ambiente utilizando um resistor dependente de luz (LDR).

Lógica de Funcionamento

O sistema atua como uma fotocélula inteligente:

- **Ambiente Iluminado:** O sensor detecta alta luminosidade e mantém o LED **apagado** para economizar energia.
- **Ambiente Escuro:** Ao detectar a ausência de luz (sensor coberto), o LED **acende automaticamente**.
- **Aplicação Prática:** Esta é a mesma lógica utilizada em postes de iluminação pública e luzes de emergência.
- **Desafio Analógico:** Aprender a definir o "ponto de corte"(valor de brilho) para a ativação do LED.



	01	Placa Uno com cabo usb
	01	Protoboard
	01	Led
	01	Resistor 150R (marrom, verde, marrom, dourado)
	01	LDR (sensor de luminosidade)
	01	Resistor 10K (marrom, preto, laranja, dourado)
	06	Jumper macho x macho

Projeto 7: Código Fonte (pyfirmata2)

Objetivo: Acender um LED automaticamente ao detectar baixa luminosidade.

```
from pyfirmata2 import Arduino
import time

placa = Arduino('COM3')
led = placa.get_pin('d:9:o')
ldr = placa.get_pin('a:5:i')

placa.samplingOn()
time.sleep(0.1)
print("Sistema Ativo. Pressione Ctrl+C para sair.")

try:
    while True:
        nivel_luz = ldr.read()
        if nivel_luz is not None:
            print(f"Luminosidade: {round(nivel_luz, 2)}")
            # Se luz < 0.5 (equivale a 500 no Arduino)
            if nivel_luz < 0.5:
                led.write(1)
            else:
                led.write(0)
            time.sleep(0.5)
except KeyboardInterrupt:
    placa.exit()
```

Encerramento
